

Structuring the Cinematic Record

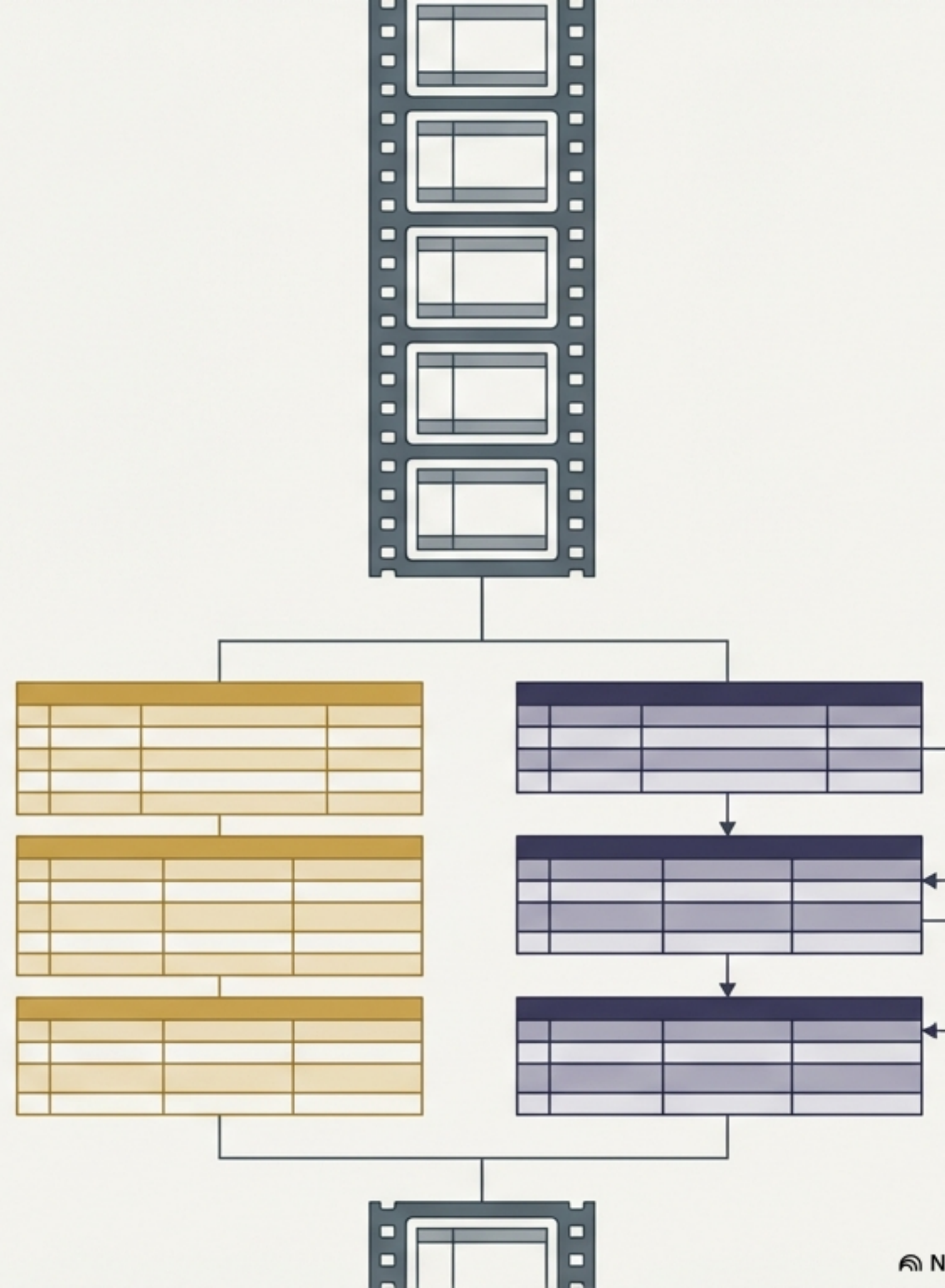
A Dimensional Modeling Pipeline

Project Brief

Project Focus: Transforming the transactional actor_films dataset into an optimized analytical engine using PostgreSQL.

Core Technologies: Slowly Changing Dimensions (SCD Type 2), DDL formulation, Cumulative Table Generation, Window Functions.

Pipeline Strategy: Backfill historical processing (1971–2020) and Incremental updating (2021).



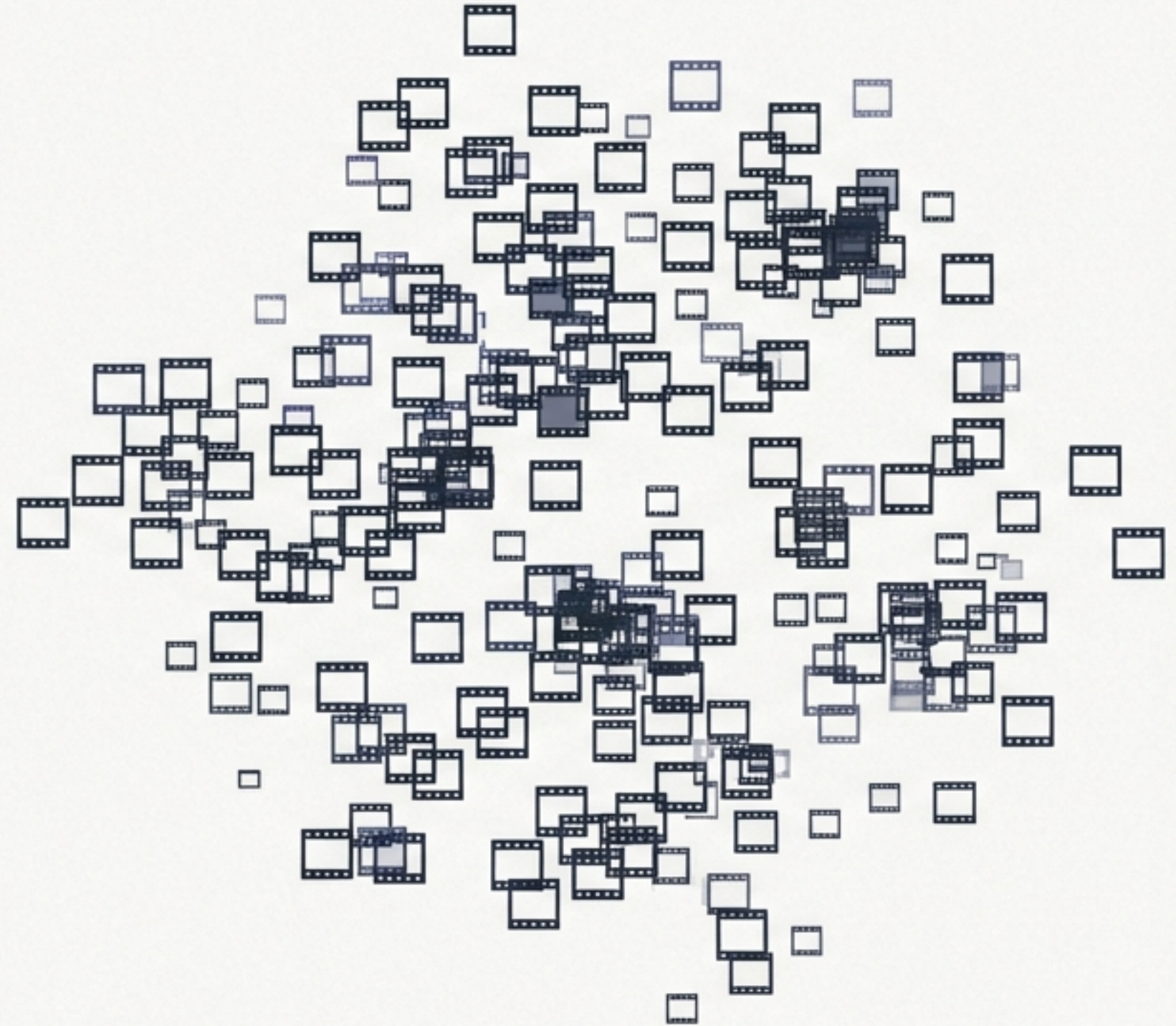
Raw Transactional Data Fragments Historical Analysis

actor_films
PK: actorid, filmid actor year votes rating

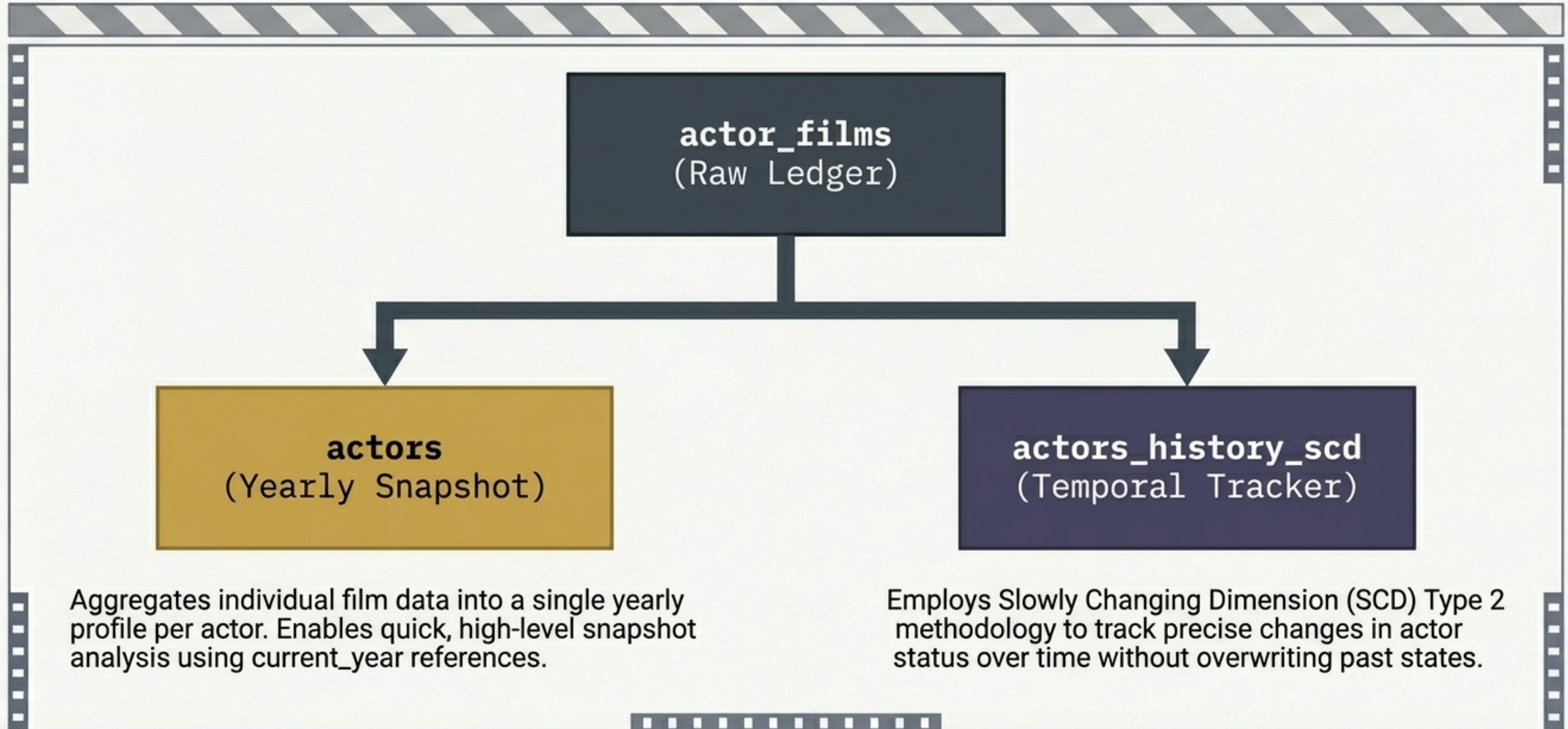
The Baseline Dataset: A comprehensive ledger of films, actors, and ratings spanning from 1970 to 2021.

The Analytical Bottleneck: Transactional data logs events year-by-year. It lacks a centralized mechanism to track an actor's aggregated yearly performance or long-term career trajectory.

The Objective: Synthesize this raw ledger into a structured dimensional model to unlock rapid, temporal insights.



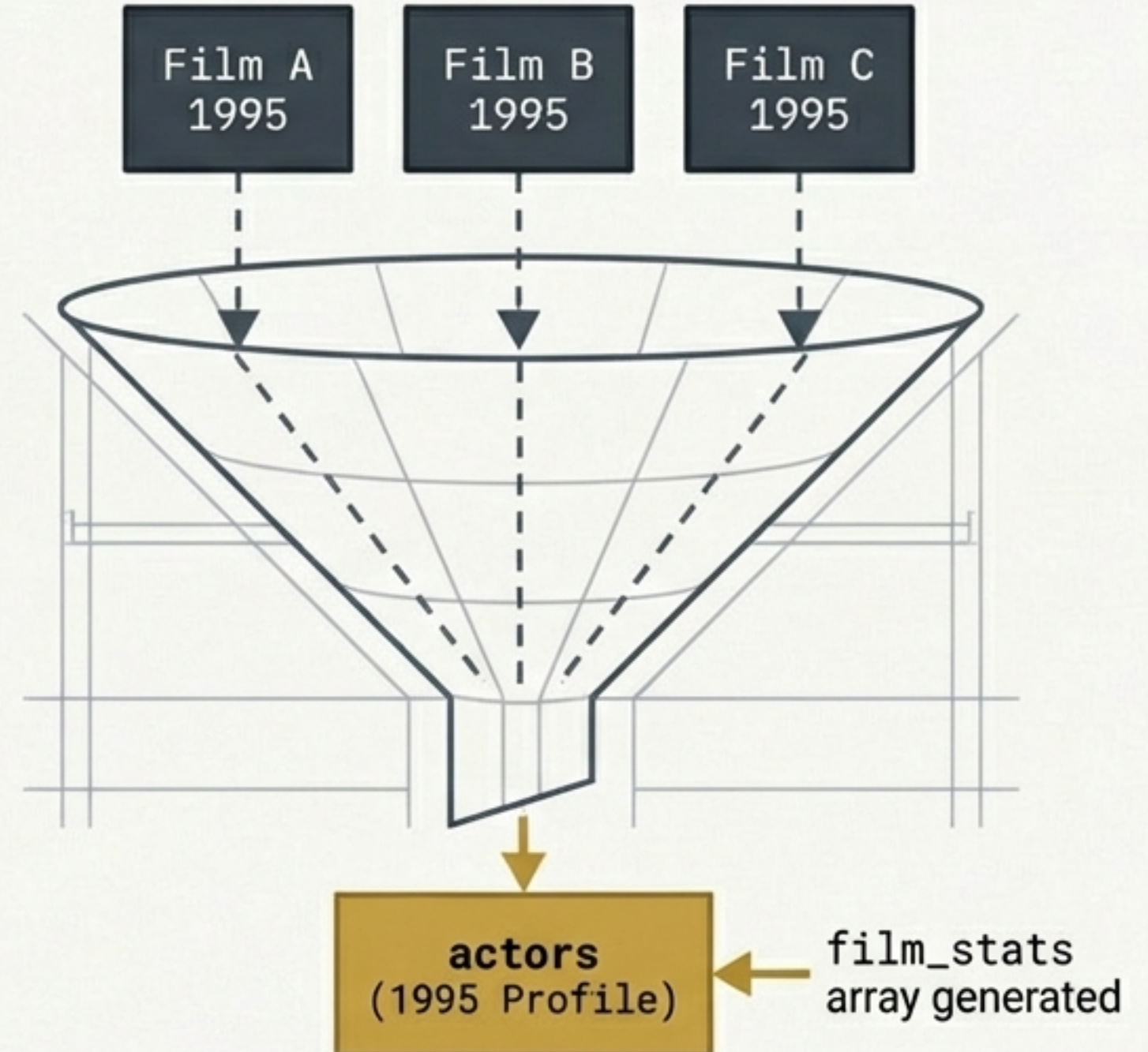
Designing the Dual-Table Dimensional Architecture



Compressing Transactional Noise into Yearly Snapshots

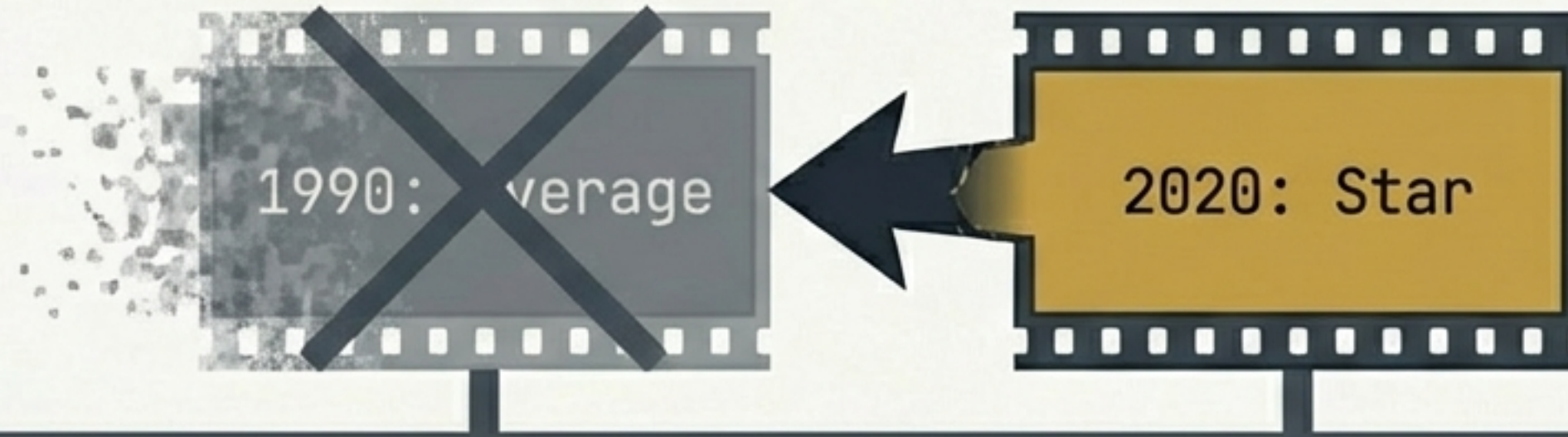
Mechanics

- **Cumulative Generation:** Iteratively populates actor profiles year by year from 1971 to 2021.
- **Custom Types Formulated:** Utilizes array types `film_stats` to group movie details and `quality_check` to categorize ratings (star/good/average/bad).
- **Result:** A fully aggregated view of an actor's output and quality classification for any given `current_year`.



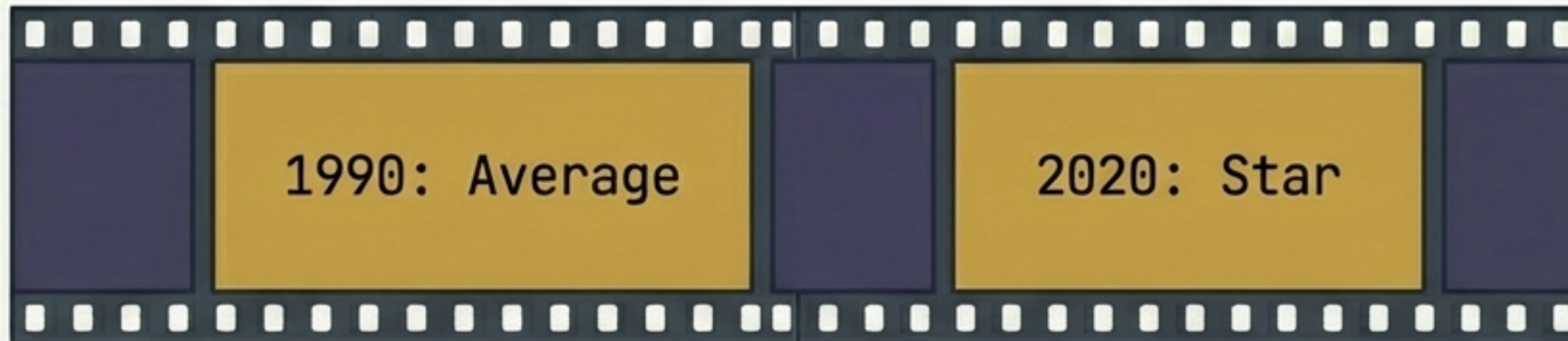
The Imperative of Preserving Historical Context

The State Tracking Dilemma (Standard Overwrite)



When an actor's `quality_class` or `is_active` status changes, standard row updates destroy the historical record.

The Requirement (Historical Preservation)



To perform accurate trend analysis, the database must retain multiple validity periods. We close an old record and open a new one precisely when a status shift occurs.

Implementing SCD Type 2 for Continuous Career Tracking



quality_class	start_date	end_date
quality_class: average	start_date: 1990	end_date: 2005
quality_class: good	start_date: 2005	end_date: 2020
quality_class: star	start_date: 2020	end_date: 9999

Temporal Boundaries:

Utilizes explicit **start_date** and **end_date** columns to define the precise lifespan of any given record.

Non-Destructive Updates:

Historical changes trigger the addition of new rows rather than data mutations, creating an immutable audit trail.

Post-Production Data Loading: Strategic Alignment

	Strategy: BACKFILL	Strategy: INCREMENTAL
Cinematic Metaphor	Filming the entire movie in one go, capturing the full backstory.	Shooting a new scene to continue the story.
Target Era	1971 – 2020	2021 Data Update
Primary Use Case	Initializing a new table, catching up on historical data, rebuilding post-loss.	Adding periodic updates (e.g., daily, yearly) efficiently.
Pros	Comprehensive, ensures total dataset consistency.	Highly efficient, faster execution, scalable for ongoing operations.
Cons	Resource-intensive, slow for large volumes, risks redundant processing.	Requires pre-existing history, dependency management is complex.

Reconstructing History via Comprehensive Backfilling



The Operation

Scanning all historical data from scratch to generate complete SCD records up to the year 2020.

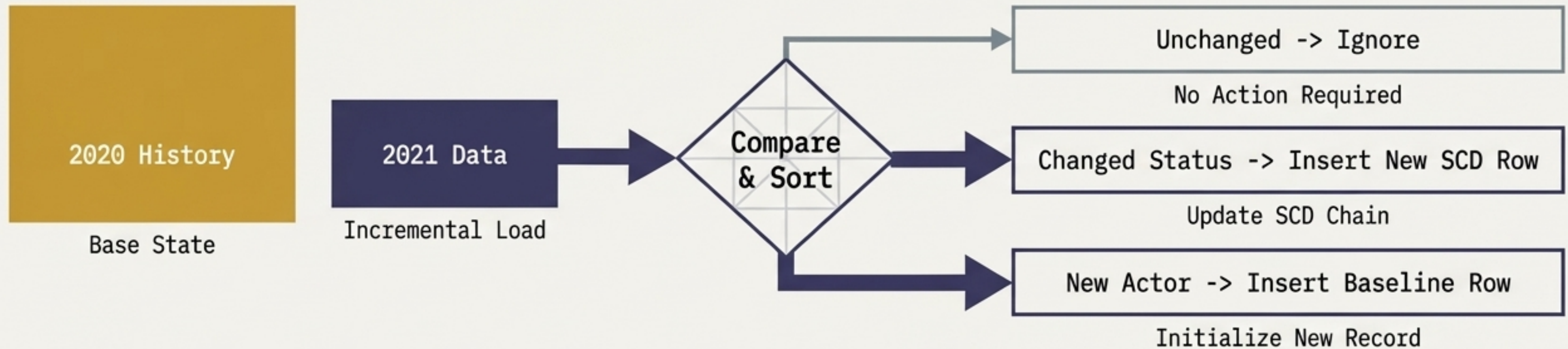
Change Detection

Leverages PostgreSQL `WINDOW_FUNCTION()` to examine adjacent rows, mathematically detecting precisely when an actor's **quality_class** or **is_active** state shifts.

Outcome

Establishes the foundational truth of the database, readying the system for lightweight future updates.

Extending the Timeline via Incremental Processing



The Operation

At the conclusion of 2021, the pipeline compares the new actors dataset against the 2020 baseline.

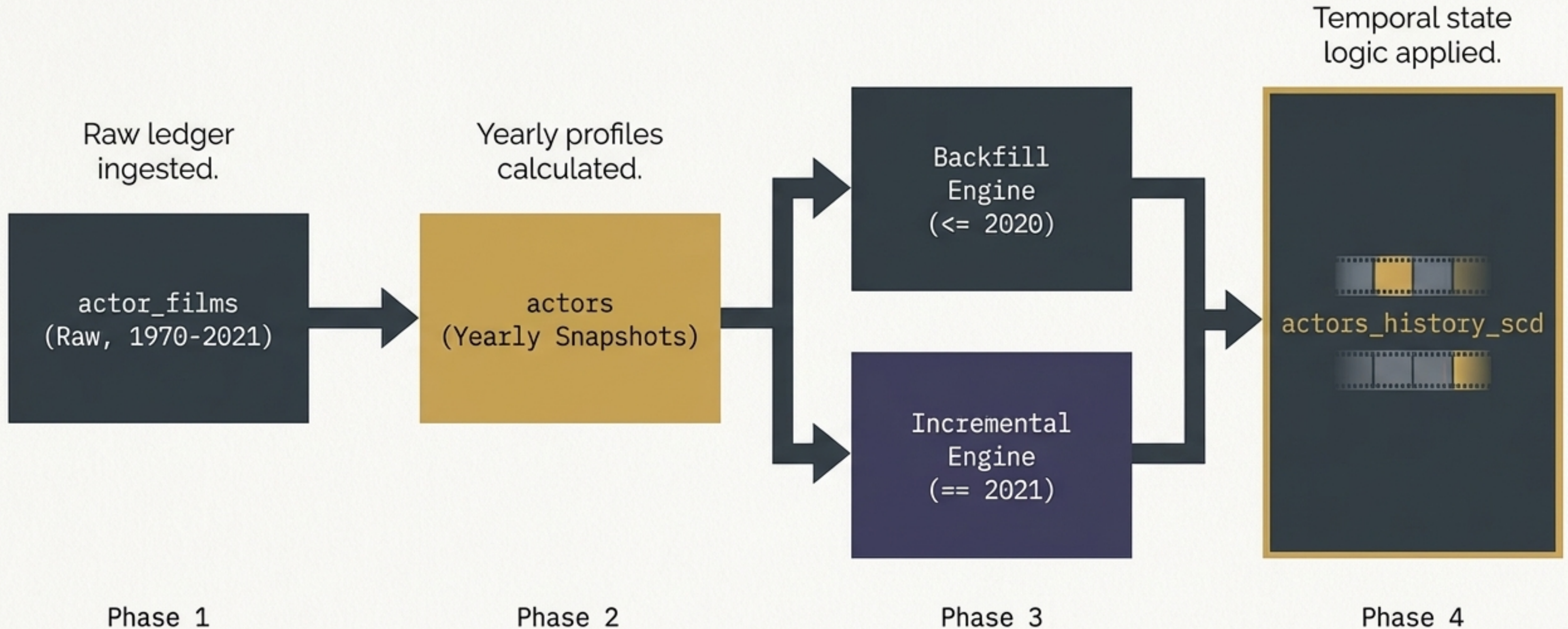
Precision Insertion

Identifies deltas in the dataset. It only processes and inserts new rows for actors whose status fundamentally changed or who made their screen debut in 2021.

Outcome

Keeps the **actors_history_scd** table perfectly synced without the computational overhead of scanning 50 years of data.

The Unified Data Lifecycle Architecture



Project Outcomes and Technical Validation

Built Dimensional Data Model

Designed and deployed the SQL-based `actors_history_scd` and `actors` tables, radically optimizing the `actor_films` dataset for rapid performance analysis.

Implemented SCD Type 2 Logic

Authored complex backfill and incremental PostgreSQL queries to track decades of actor quality and activity fluctuations (1971-2021).

System Impact

Eliminated the analytical constraints of raw transactional data, establishing a scalable, history-preserving architecture ready for modern BI workloads.

